

Insurance Management System

A Report Submitted in

Partial Fulfilment for

award of

Bachelor of Technology

In

**ADVANCED JAVA PROGRAMMING
(BCSE0651/BCSEH0651/BMICSE0651)**

COMPUTER SCIENCE & ENGINEERING

By

DRISHAY CHAUHAN (2301330100084)

HIMANSHI SHARMA (2301330100095)

Under the Supervision of

Dr. Surya Prakash Sharma

Assistant Professor



Department of Computer Science & Engineering

School of Computer Science and Information Technology

NOIDA INSTITUTE OF ENGINEERING AND TECHNOLOGY

GREATER NOIDA

(An Autonomous Institute)

Affiliated to

DR. A.P.J. ABDUL KALAM TECHNICAL UNIVERSITY, LUCKNOW

2025-26

DECLARATION

We hereby declare that the work presented in this report was carried out by me. We have not submitted the matter embodied in this report for the award of any degree or diploma of any other University or Institute.

Team Leader:

Drishay Chauhan (2301330100084)

(Candidate's Signature)

Himanshi Sharma (2301330100095)

(Candidate's Signature)

CERTIFICATE

Certified that **Drishay Chauhan (2301330100084)** has carried out the project work presented in this Project Report in partial fulfillment of the requirements for the award of **Bachelor of Technology, CSE 3rd Year, 6th Sem** from **Noida Institute of Engineering & Technology** under my supervision.

Signature: _____

Dr. Tripti Sharma

HOD

CSE & M.TECH INT.

NIET, Gr. Noida

ACKNOWLEDGEMENT

I would like to express my heartfelt gratitude to Dr. Surya Prakash Sharma for his invaluable guidance, support, and constant supervision throughout the development of this project. Their expertise and encouragement have been pivotal in shaping the project.

I would also like to extend my thanks and appreciation to our esteemed **HOD (Dr. Tripti Sharma)** for her continuous motivation and support. Their encouragement has been instrumental in completing this project successfully.

Finally, I am deeply thankful to my peers and everyone who directly or indirectly contributed to this project with their knowledge, advice, and assistance.

ABSTRACT

The Insurance Management System is a web-based application developed using Java and the Spring Boot framework to streamline and automate the management of insurance-related operations. The primary objective of the system is to provide a centralized platform where users can easily interact with insurance services, including viewing available policies, purchasing suitable plans, and raising claims, while administrators can efficiently manage policies and handle claim processing.

The system is designed with a role-based access mechanism, ensuring clear separation between user and administrator functionalities. Users can register and log in securely, browse different insurance policies, and purchase them based on their requirements. Additionally, users can track their active policies and submit claims, which are then reviewed by the administrator. On the other hand, administrators have full control over the system, including adding new policies, deleting existing ones, and approving or rejecting claims submitted by users.

The backend of the application is built using Spring Boot and follows the Model-View-Controller (MVC) architecture, which ensures a structured and maintainable codebase. Spring Data JPA is used for database interaction, enabling efficient storage, retrieval, and management of data such as users, policies, subscriptions, and claims. The frontend is developed using JSP and styled with Bootstrap, providing a user-friendly and responsive interface.

To enhance reliability and data integrity, several validation mechanisms have been implemented. These include preventing duplicate policy purchases, ensuring that only authorized users can perform specific actions, and restricting deletion of policies that are already associated with active user subscriptions. Session management is used to maintain user authentication throughout the application.

Overall, the Insurance Management System demonstrates the practical application of modern web development technologies and design principles. It serves as a functional prototype of a real-world insurance platform and can be further enhanced by incorporating advanced features such as secure authentication mechanisms, payment integration, and scalability improvements in future versions.

TABLE OF CONTENTS

	Page No.
Declaration	i
Certificate	ii
Acknowledgement	iii
Abstract	iv
CHAPTER 1: INTRODUCTION	1
1.1 Overview	
1.2 Objective And Scope	
CHPATER 2: LITERATURE REVIEW	3
2.1 Overview of Existing Systems	
2.2 Limitations of Existing Systems	
CHAPTER 3: REQUIREMENTS	5
3.1 Functional Requirements	
3.2 Non-Functional Requirements	
3.3 Software and Hardware Requirements	
CHAPTER 4: SYSTEM DESGIN AND IMPLEMENTATION	7
4.1 System Architecture (Flow diagram)	
4.2 GUI Design	
4.3 Database Design	
4.4 Implementation Details	
CHAPTER 5: TESTING AND RESULTS	10
5.1 Testing Methodology	
5.2 Test Cases and Results	
CHAPTER 6: CONCLUSION AND FUTURE WORK	12
6.1 Conclusion	
6.2 Future Work	
APPENDIX	14

INTRODUCTION

1.1 Overview

The Insurance Management System is a web-based application developed to simplify and digitize the process of managing insurance services. Traditionally, insurance-related operations such as policy selection, purchase, and claim processing involve manual procedures, paperwork, and delays. This project aims to overcome these limitations by providing an efficient and user-friendly digital platform.

The system is built using Java and the Spring Boot framework, following the Model-View-Controller (MVC) architecture. It allows users to register, log in, view available insurance policies, purchase policies, and raise claims through an interactive web interface. At the same time, administrators are provided with dedicated functionalities to manage policies and handle user claims effectively.

The application uses Spring Data JPA for database interaction, ensuring efficient data management and persistence. The frontend is developed using JSP and Bootstrap, offering a clean and responsive user experience. Role-based access control is implemented to ensure that users and administrators have appropriate permissions and secure access to system functionalities.

Overall, the system provides a structured and automated approach to insurance management, reducing manual effort and improving accuracy, efficiency, and accessibility.

1.2 Objective and Scope

Objective:

The primary objectives of the Insurance Management System are:

- To develop a web-based platform for managing insurance policies and claims.
- To enable users to easily browse, purchase, and track insurance policies.
- To provide a system for users to raise and monitor claims efficiently.
- To allow administrators to manage policies, including adding and deleting policies.
- To implement a role-based system ensuring secure and controlled access.
- To ensure data integrity by preventing duplicate purchases and unauthorized actions.
- To demonstrate the practical use of Spring Boot, JPA, and MVC architecture.

Scope:

The scope of the project includes:

- Development of a user-friendly interface for both users and administrators.
- Implementation of authentication and session management for secure access.
- Management of insurance policies, including viewing, purchasing, and deletion.
- Handling of claim submission and processing by administrators.
- Use of a relational database for storing user, policy, subscription, and claim data.
- Providing a basic but functional system that simulates real-world insurance operations.

However, the current system does not include advanced features such as online payment integration, real-time notifications, or advanced security frameworks. These features can be incorporated in future enhancements to make the system more scalable and production-ready.

LITERATURE REVIEW

2.1 Overview of Existing Systems

Existing insurance management systems are widely used in the insurance industry to manage policies, customer data, and claim processing. Traditionally, these systems were manual or semi-digital, relying heavily on paperwork, human intervention, and offline processes. Over time, many organizations have shifted to digital platforms, including web-based and enterprise-level systems, to improve efficiency and accessibility.

Modern insurance systems provide features such as policy management, premium calculation, claim tracking, and customer relationship management. Large-scale systems used by insurance companies are often complex and integrated with multiple services such as payment gateways, fraud detection, and analytics tools. These systems are usually built using enterprise technologies and are designed to handle large volumes of data and users.

However, many existing systems are either too complex for small-scale applications or lack user-friendly interfaces. Some systems focus more on administrative control and provide limited interaction for end users. In academic or small-scale implementations, systems are often simplified and may not fully demonstrate real-world functionalities such as role-based access, structured data handling, and proper workflow management.

2.2 Limitations of Existing Systems

Despite advancements, existing systems have several limitations, especially in simpler or traditional implementations:

- **Manual Processes:** Many traditional systems rely on manual data entry and paperwork, leading to delays and increased chances of human error.
- **Limited Accessibility:** Offline or semi-digital systems restrict users from accessing services anytime and anywhere.
- **Complex Interfaces:** Some modern systems are overly complex, making them difficult for users to navigate and understand.
- **Lack of Role-Based Control:** Basic systems often do not implement proper role-based access, leading to security concerns and unauthorized actions.
- **Inefficient Claim Processing:** Claim handling in traditional systems can be slow and lacks transparency for users.
- **Poor Data Integrity:** Absence of validation mechanisms may lead to duplicate entries or inconsistent data.

- Scalability Issues: Smaller systems are not designed to handle growth or additional features efficiently.
- Security Limitations: Traditional systems may lack advanced authentication and protection mechanisms for sensitive user data.
- Limited Automation: Many existing systems do not provide automated notifications, policy tracking, or streamlined workflows.

REQUIREMENTS

3.1 Functional Requirements

Functional requirements define what the system is expected to do. The key functional requirements of the Insurance Management System are:

- The system shall allow users to register and create an account.
- The system shall allow users to log in using valid credentials.
- The system shall provide role-based access for users and administrators.
- The system shall display a list of available insurance policies.
- The system shall allow users to purchase insurance policies.
- The system shall prevent users from purchasing the same policy multiple times.
- The system shall allow users to view their purchased policies.
- The system shall allow users to raise claims for their policies.
- The system shall allow users to view the status of their claims.
- The system shall allow administrators to add new insurance policies.
- The system shall allow administrators to delete policies (if not in use).
- The system shall allow administrators to view all user claims.
- The system shall allow administrators to approve or reject claims.
- The system shall maintain session-based authentication for users.
- The system shall allow users to log out securely.

3.2 Non-Functional Requirements.

Non-functional requirements define how the system performs and behaves.

- **Performance:** The system should respond quickly to user requests and handle multiple operations efficiently.
- **Usability:** The user interface should be simple, intuitive, and easy to navigate.
- **Reliability:** The system should operate consistently without failures during normal usage.
- **Security:** Only authorized users should access specific functionalities using role-based authentication.
- **Scalability:** The system should be designed in a way that allows future enhancements and scaling.
- **Maintainability:** The code structure should follow MVC architecture to ensure easy maintenance and updates.
- **Data Integrity:** The system should ensure accurate and consistent data through validations.
- **Availability:** The system should be accessible anytime through a web interface.

3.3 Software and Hardware Requirements

Software Requirements:

- **Programming Language:** Java
- **Framework:** Spring Boot
- **Architecture:** Model-View-Controller (MVC)
- **Database:** MySQL
- **ORM Tool:** Spring Data JPA / Hibernate
- **Frontend Technologies:** JSP, HTML, CSS, Bootstrap
- **IDE:** Eclipse / IntelliJ IDEA
- **Build Tool:** Maven
- **Web Server:** Apache Tomcat

Hardware Requirements:

- **Processor:** Intel i3 or higher
- **RAM:** Minimum 4 GB
- **Storage:** At least 500 MB free space
- **System Type:** 64-bit Operating System
- **Internet Connection:** Required for dependencies and deployment

SYSTEM DESIGN AND IMPLEMENTATION

4.1 System Architecture

The Insurance Management System follows the **Model-View-Controller (MVC)** architecture, which separates the application into three main components:

- **Model:** Represents the data layer, including entities such as User, Policy, Subscription, and Claim. It interacts with the database using Spring Data JPA.
- **View:** Represents the presentation layer, implemented using JSP and Bootstrap, which provides the user interface.
- **Controller:** Handles user requests, processes business logic, and connects the Model and View components.

Working Flow:

1. The user sends a request through the web interface (JSP page).
2. The request is received by the Controller.
3. The Controller processes the request and interacts with the Model (database).
4. The Model returns the required data to the Controller.
5. The Controller sends the response to the View.
6. The View displays the result to the user.

4.2 GUI Design

The graphical user interface (GUI) of the system is designed to be simple, clean, and user-friendly using JSP and Bootstrap.

Key Screens:

- **Login Page:** Allows users to log in using credentials.
- **Register Page:** Enables new users to create an account.
- **User Dashboard:** Displays user-specific options such as viewing policies and claims.
- **Admin Dashboard:** Provides administrative controls like managing policies and claims.
- **Policy List Page:** Shows all available policies with options to buy (user) or delete (admin).
- **My Policies Page:** Displays policies purchased by the user.
- **Claim Page:** Allows users to raise claims.
- **Claims Management Page:** Allows admin to approve or reject claims.

The interface uses a consistent navigation bar and Bootstrap styling to ensure responsiveness and ease of use.

4.3 Database Design

The system uses a relational database (MySQL) to store and manage data efficiently.

Main Tables:

- **User Table:**
 - id, name, email, password, role
- **Policy Table:**
 - id, name, type, premium, duration, description
- **Subscription Table:**
 - id, user_id, policy_id, start_date, status
- **Claim Table:**
 - id, user_id, policy_id, amount, reason, status, date

Relationships:

- One User can have multiple Subscriptions (One-to-Many)
- One Policy can be linked to multiple Subscriptions (One-to-Many)
- One Subscription connects one User and one Policy (Many-to-One)
- Claims are associated with Users and Policies

This design ensures proper data organization and supports efficient querying.

4.4 Implementation Details

The system is implemented using the Spring Boot framework and follows best practices of web application development.

- Controllers are used to handle HTTP requests such as login, policy management, and claim processing.
- Spring Data JPA is used to interact with the database, reducing the need for complex SQL queries.
- Session management is implemented to maintain user authentication across pages.
- Role-based access control ensures that users and administrators have different permissions.
- Validation logic is implemented to:
 - Prevent duplicate policy purchases
 - Restrict unauthorized access
 - Prevent deletion of policies that are already in use

- JSP pages are used to render dynamic content, and Bootstrap is used for styling and responsive user interface design.
- MVC architecture is followed to maintain proper separation between business logic, presentation, and database operations.
- Exception handling is implemented to improve system reliability and user experience.
- The system is modular, maintainable, scalable, and can be extended with additional features such as payment integration, advanced security, and analytics.

TESTING

5.1 Testing Methodology

Testing is an essential phase in software development to ensure that the system functions correctly and meets the specified requirements. The Insurance Management System was tested using a combination of manual testing techniques to verify both functionality and usability.

The testing process involved validating different modules of the system such as user authentication, policy management, and claim processing. Each feature was tested with valid and invalid inputs to ensure proper handling of different scenarios. Special attention was given to role-based access control to verify that users and administrators can only access their respective functionalities.

The following testing approaches were used:

- **Unit Testing:** Individual components such as controllers and methods were tested independently to ensure correct behavior.
- **Integration Testing:** Interaction between modules (e.g., user and policy, policy and subscription) was tested to ensure proper data flow.
- **System Testing:** The entire system was tested as a whole to verify that all components work together seamlessly.
- **Validation Testing:** Input fields and business logic were tested to ensure data accuracy and prevent invalid operations.

The testing process ensured that the application is stable, reliable, and performs as expected under normal conditions.

5.2 Test Cases and Results

The following table represents some key test cases performed on the system:

Test Case ID	Description	Input	Expected Output	Actual Output	Status
TC01	User Registration	Valid user details	User registered successfully	Success message displayed	Pass
TC02	User Login	Correct credentials	Redirect to dashboard	Dashboard loaded	Pass
TC03	Invalid Login	Wrong credentials	Error message	Error shown	Pass
TC04	View Policies	Logged-in user	List of policies displayed	Policies shown	Pass
TC05	Buy Policy	Valid policy ID	Policy purchased	Policy added to My Policies	Pass
TC06	Duplicate Purchase	Same policy again	Purchase blocked	Redirect without duplication	Pass

TC07	Raise Claim	Valid inputs	Claim submitted	Claim recorded	Pass
TC08	View Claims	Logged-in user	List of claims	Claims displayed	Pass
TC09	Admin Add Policy	Valid inputs	Policy added	Policy visible in list	Pass
TC10	Delete Policy (Unused)	Valid policy ID	Policy deleted	Policy removed	Pass
TC11	Delete Policy (In Use)	Linked policy	Deletion blocked	Error shown	Pass
TC12	Approve Claim	Admin action	Status updated	Claim approved	Pass
TC13	Unauthorized Access	Direct URL	Access denied	Redirect to login	Pass
TC14	Logout	Click logout	Session ended	Redirect to login	Pass

CONCLUSION AND FUTURE WORK

6.1 Conclusion

The Insurance Management System developed in this project successfully demonstrates the implementation of a web-based application for managing insurance services in an efficient and structured manner. The system provides a user-friendly interface for both users and administrators, enabling smooth interaction with different functionalities such as policy management, policy purchase, and claim processing.

The application effectively eliminates the need for manual processes by automating key operations like registration, authentication, and claim handling. The use of the Spring Boot framework and MVC architecture ensures a well-organized and maintainable code structure. Integration with a relational database using Spring Data JPA allows efficient data storage and retrieval.

The system also incorporates important validation mechanisms such as preventing duplicate policy purchases and restricting the deletion of policies that are already in use. Role-based access control ensures that users and administrators have secure and appropriate access to system features.

Overall, the project achieves its objective of building a functional and reliable insurance management platform. It demonstrates practical knowledge of modern web development technologies and provides a strong foundation for developing more advanced and scalable systems.

6.2 Future Work

Although the current system is fully functional, several enhancements can be implemented to improve its capabilities and make it more suitable for real-world deployment:

- **Secure Authentication:** Integration of Spring Security and password encryption for enhanced security.
- **Payment Integration:** Adding online payment functionality for purchasing insurance policies.
- **Email Notifications:** Sending automated emails for policy confirmation and claim updates.
- **Advanced UI/UX:** Improving the interface using modern frontend frameworks like React or Angular.
- **Search and Filter Options:** Allowing users to easily search and filter policies.
- **Pagination:** Managing large data sets efficiently using pagination.
- **Admin Analytics Dashboard:** Providing insights and reports for better decision-making.
- **Cloud Deployment:** Deploying the system on cloud platforms such as AWS or Azure.
- **Mobile Compatibility:** Developing a mobile application version for better accessibility.

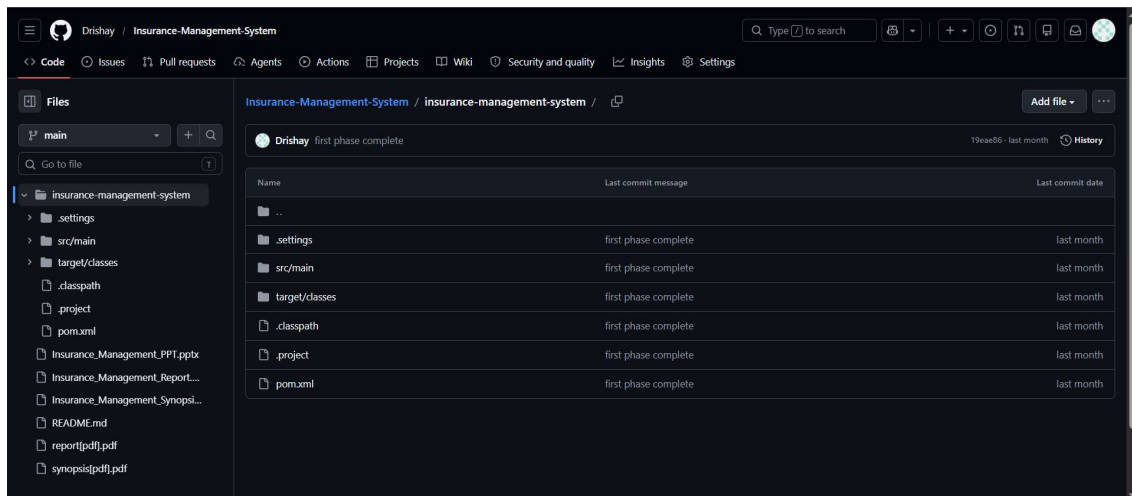
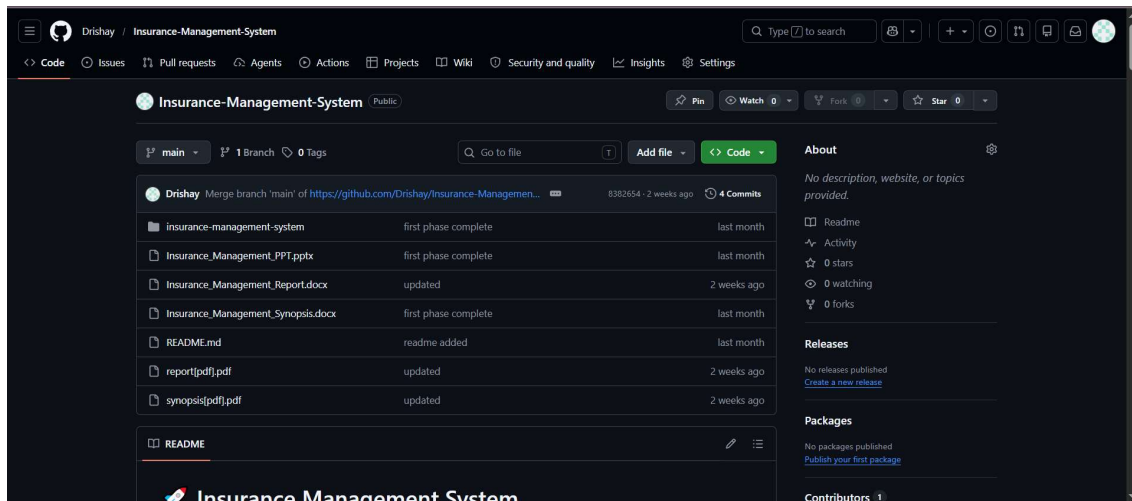
These improvements can significantly enhance the usability, security, and scalability of the system, making it closer to a real-world insurance platform.

APPENDIX

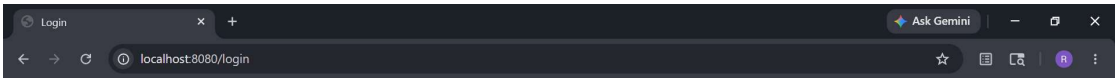
Source Code: <https://github.com/Drishay/Insurance-Management-System>

Snapshots

GITHUB repo structure



LOGIN / SIGNUP



Login

Email

Password

Login

New user?
[Create Account](#)

Create Account

Name

Email

Password

Register

Already have an account?
[Login](#)

USER Dashboard

Login

Email

Test1@gmail.com

Password

....

Login

New user?

Create Account

Welcome, test1

HomePoliciesMy PoliciesRaise ClaimMy ClaimsMy AccountLogout

Welcome, test1@gmail.com

Policies

Browse available insurance plans.

View

Policies

5

My Policies

Check your purchased policies.

Open

My Policies

2

Claims

Raise or track claims.

Check

Claims

4

All Policies

[Home](#) [Policies](#) [My Policies](#) [Raise Claim](#) [My Claims](#) [My Account](#) [Logout](#)

User: test1@gmail.com

ID	Name	Type	Premium	Duration	Action
1	Health Secure Plan	Health Insurance	5000.0	12	Purchased
2	Health Secure Plan	Health	5000.0	12	Buy
3	Life Protect Plan	Life	8000.0	20	Purchased
5	Family Health Plus	Health	7000.0	12	Buy
6	Travel Safe Insurance	Travel	2000.0	6	Buy

My Policies

[Home](#) [Policies](#) [My Policies](#) [Raise Claim](#) [My Claims](#) [My Account](#) [Logout](#)

Welcome, test1@gmail.com

Policy	Status	Start Date
Health Secure Plan	ACTIVE	2026-01-01
Life Protect Plan	ACTIVE	2026-02-01

Raise Claim

[Home](#) [Policies](#) [My Policies](#) [Raise Claim](#) [My Claims](#) [My Account](#) [Logout](#)

Welcome, test1@gmail.com

Policy

Health Secure Plan

Amount

Reason

[Submit Claim](#)

My Account

[Home](#) [Policies](#) [My Policies](#) [Raise Claim](#) [My Claims](#) [My Account](#) [Logout](#)

Welcome, **test1@gmail.com**

Name: test1

Email: test1@gmail.com

Role: USER

[Back](#)

My Claims

[Home](#) [Policies](#) [My Policies](#) [Raise Claim](#) [My Claims](#) [My Account](#) [Logout](#)

Welcome, **test1@gmail.com**

Policy	Amount	Reason	Status	Date
Health Secure Plan	2000.0	Hospital expenses	REJECTED	2026-03-01
Life Protect Plan	1500.0	Car accident repair	REJECTED	2026-03-05
Health Secure Plan	1230.0	acc	APPROVED	2026-04-24
Life Protect Plan	33000.0	ghumne jana hai	APPROVED	2026-04-24

ADMIN Dashboard

Login

Email

drishay@gmail.com

Password

...

Login

New user?

Create Account

Admin Dashboard

Admin Panel

Home

Policies

Add Policy

Manage Claims

My Account

Logout

Admin: drishay@gmail.com

Policies

View and manage all policies.

View Policies

Add Policy

Create new insurance policies.

Add

Claims

Approve or reject user claims.

Manage

All Policies

[Home](#)[Policies](#)[Add Policy](#)[Manage Claims](#)[My Account](#)[Logout](#)

Admin: drishay@gmail.com

ID	Name	Type	Premium	Duration	Action
1	Health Secure Plan	Health Insurance	5000.0	12	Delete
2	Health Secure Plan	Health	5000.0	12	Delete
3	Life Protect Plan	Life	8000.0	20	Delete
5	Family Health Plus	Health	7000.0	12	Delete
6	Travel Safe Insurance	Travel	2000.0	6	Delete

Add Policy

Admin Panel

[Home](#)[Policies](#)[Add Policy](#)[Manage Claims](#)[My Account](#)[Logout](#)

Admin: drishay@gmail.com

Name

Type

Premium

Duration

Description

[Save Policy](#)

Manage Claims

Admin Panel					
Admin: drishay@gmail.com					
ID	User	Policy	Amount	Status	Action
1	test1@gmail.com	Health Secure Plan	2000.0	REJECTED	Approve Reject
2	test1@gmail.com	Life Protect Plan	1500.0	REJECTED	Approve Reject
3	admin@gmail.com	Health Secure Plan	5000.0	REJECTED	Approve Reject
4	test1@gmail.com	Health Secure Plan	1230.0	APPROVED	Approve Reject
5	test1@gmail.com	Life Protect Plan	33000.0	APPROVED	Approve Reject

Admin Account

Admin Panel					
Admin: drishay@gmail.com					
Name: Drishay Email: drishay@gmail.com Role: ADMIN					
Back					

DATABASE – SQL

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

insurance_db

Tables

claim

policy

subscription

user

Views

Stored Procedures

Functions

Information

Table: claim

Columns:

id int AI PK

amount double

date date

reason varchar(255)

status varchar(255)

policy_id int

user_id int

SQL File 1 claim

1 • SELECT * FROM insurance_db.claim;

Limit to 1000 rows

Result Grid

	id	amount	date	reason	status	policy_id	user_id
1	1	2000	2026-03-01	Hospital expenses	REJECTED	1	2
2	2	1500	2026-03-05	Car accident repair	REJECTED	3	2
3	3	5000	2026-03-10	Life insurance claim	REJECTED	2	3
4	4	1230	2026-04-24	acc	APPROVED	1	2
5	5	33000	2026-04-24	ghumne jana hai	APPROVED	3	2

SQLAdditions

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Context Help Snippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	19:56:04	SELECT * FROM insurance_db.claim LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

insurance_db

Tables

claim

policy

subscription

user

Views

Stored Procedures

Functions

Information

Table: policy

Columns:

id int AI PK

description varchar(255)

duration int

name varchar(255)

premium double

type varchar(255)

SQL File 1 claim policy

1 • SELECT * FROM insurance_db.policy;

Limit to 1000 rows

Result Grid

	id	description	duration	name	premium	type
1	1	Covers hospitalization, medicines, and emergen...	12	Health Secure Plan	5000	Health Insurance
2	2	Covers hospitalization and medical expenses	12	Health Secure Plan	5000	Health
3	3	Provides life coverage and maturity benefits	20	Life Protect Plan	8000	Life
5	5	Covers entire family health expenses	12	Family Health Plus	7000	Health
6	6	Covers travel-related risks	6	Travel Safe Insurance	2000	Travel

SQLAdditions

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Context Help Snippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	19:56:04	SELECT * FROM insurance_db.claim LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec
2	19:56:29	SELECT * FROM insurance_db.policy LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

insurance_db

Tables

claim

policy

subscription

user

Views

Stored Procedures

Functions

java

sakila

sales

sessional

starter_db

Administration

Schemas

Information

Table: subscription

Columns:

id int AI PK

start_date date

status varchar(255)

policy_id int

user_id int

Object Info

Session

SQL File 1 claim policy subscription

Limit to 1000 rows

1 • SELECT * FROM insurance_db.subscription;

Result Grid

	id	start_date	status	policy_id	user_id
1	1	2026-01-01	ACTIVE	1	2
2	2	2026-02-01	ACTIVE	3	2
3	3	2026-01-15	ACTIVE	2	3
4	NULL	NULL	NULL	NULL	NULL

subscription 1 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	19:56:04	SELECT * FROM insurance_db.claim LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec
2	19:56:29	SELECT * FROM insurance_db.policy LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec
3	19:56:49	SELECT * FROM insurance_db.subscription LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec

SQLAdditions

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

insurance_db

Tables

claim

policy

subscription

user

Views

Stored Procedures

Functions

java

sakila

sales

sessional

starter_db

Administration

Schemas

Information

Table: user

Columns:

id int AI PK

email varchar(255)

name varchar(255)

password varchar(255)

role varchar(255)

Object Info

Session

SQL File 1 claim policy subscription user

Limit to 1000 rows

1 • SELECT * FROM insurance_db.user;

Result Grid

	id	email	name	password	role
1	1	drishay@gmail.com	Drishay	123	ADMIN
2	2	test1@gmail.com	test1	123	USER
3	3	admin@gmail.com	Admin User	admin123	ADMIN
4	4	rahul@gmail.com	Rahul Sharma	1234	USER
5	5	priya@gmail.com	Priya Singh	1234	USER
6	NULL	NULL	NULL	NULL	NULL

user 1 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	19:56:04	SELECT * FROM insurance_db.claim LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec
2	19:56:29	SELECT * FROM insurance_db.policy LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec
3	19:56:49	SELECT * FROM insurance_db.subscription LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec
4	19:57:08	SELECT * FROM insurance_db.user LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec

SQLAdditions

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

REFERENCES

- [1] Oracle Corporation, “Java Documentation,” [Online]. Available: <https://docs.oracle.com/javase/>. [Accessed: Apr. 2026].
- [2] Pivotal Software, “Spring Boot Documentation,” [Online]. Available: <https://spring.io/projects/spring-boot>. [Accessed: Apr. 2026].
- [3] Spring, “Spring Data JPA Reference Documentation,” [Online]. Available: <https://docs.spring.io/spring-data/jpa/docs/current/reference/html/>. [Accessed: Apr. 2026].
- [4] Oracle Corporation, “MySQL Documentation,” [Online]. Available: <https://dev.mysql.com/doc/>. [Accessed: Apr. 2026].
- [5] Bootstrap Team, “Bootstrap Documentation,” [Online]. Available: <https://getbootstrap.com/docs/>. [Accessed: Apr. 2026].
- [6] TutorialsPoint, “Spring Boot Tutorial,” [Online]. Available: https://www.tutorialspoint.com/spring_boot/. [Accessed: Apr. 2026].
- [7] Oracle Corporation, “Java EE (JSP & Servlet) Documentation,” [Online]. Available: <https://docs.oracle.com/javaee/>. [Accessed: Apr. 2026].
- [8] Various Authors, “Web Development Tutorials and Projects,” GitHub and Medium, [Online]. Available: <https://github.com/> and <https://medium.com/>. [Accessed: Apr. 2026].
- [9] Hibernate, “Hibernate ORM Documentation,” [Online]. Available: <https://hibernate.org/orm/documentation/>. [Accessed: Apr. 2026].
- [10] Apache Software Foundation, “Apache Maven Project Documentation,” [Online]. Available: <https://maven.apache.org/guides/>. [Accessed: Apr. 2026].
- [11] Baeldung, “Spring Framework Tutorials,” [Online]. Available: <https://www.baeldung.com/>. [Accessed: Apr. 2026].
- [12] GeeksforGeeks, “Java and Spring Boot Tutorials,” [Online]. Available: <https://www.geeksforgeeks.org/>. [Accessed: Apr. 2026].
- [13] Spring, “Spring Security Documentation,” [Online]. Available: <https://spring.io/projects/spring-security>. [Accessed: Apr. 2026].
- [14] Mozilla Developer Network (MDN), “Web Development Documentation,” [Online]. Available: <https://developer.mozilla.org/>. [Accessed: Apr. 2026].
- [15] Eclipse Foundation, “Eclipse IDE Documentation,” [Online]. Available: <https://www.eclipse.org/documentation/>. [Accessed: Apr. 2026].

- [16] Oracle Corporation, “JDBC Documentation,” [Online]. Available: <https://docs.oracle.com/javase/tutorial/jdbc/>. [Accessed: Apr. 2026].
- [17] W3Schools, “HTML, CSS, JavaScript and JSP Tutorials,” [Online]. Available: <https://www.w3schools.com/>. [Accessed: Apr. 2026].
- [18] JetBrains, “IntelliJ IDEA Documentation,” [Online]. Available: <https://www.jetbrains.com/idea/documentation/>. [Accessed: Apr. 2026].
- [19] IBM, “Database Management System Concepts,” [Online]. Available: <https://www.ibm.com/topics/database-management>. [Accessed: Apr. 2026].
- [20] Red Hat, “REST API Development Guide,” [Online]. Available: <https://www.redhat.com/en/topics/api/what-is-a-rest-api>. [Accessed: Apr. 2026].